

AI SW·어플리케이션 품질검증 특강 · 2일차

AI 어플리케이션 품질검증 프레임워크 실습

테스트 자동화 도구와 AI 모델 검증 라이브러리로 하는 품질 보증

2026. 10. 2. (금) 09:00 ~ 18:00

오늘의 흐름

2일차 시간표

교시	시간	주제	실습
1	09:00	테스트 자동화와 unittest	fork, 저장소 준비
2	10:00	PyTest: fixture, parametrize, 커버리지	경계값 테스트
3	11:00	ML 코드 테스트: 데이터·모델 테스트	누수·지표 테스트
4	13:00	교차 검증	노트북 02
5	14:00	성능 리포트 자동 생성	리포트 검토
6	15:00	CI/CD 연동 테스트 (GitHub Actions)	빨간 불 켜기
7	16:00	버그 리포팅과 이슈 트래커	이슈 → PR → 병합
8	17:00	종합 발표와 정리	조별 발표

오늘의 미션

테스트 24개 통과, 커버리지 98%.
그런데 결함이 숨어 있습니다.

1일차의 기준으로 테스트를 추가해 찾아내고, CI로 막고, 이슈로 보고하고, PR로 고칩니다.

mlqa: 유방암 데이터로 악성 종양을 찾는 작은 ML 프로젝트

파일	역할
preprocess.py	대상 판정 is_eligible(), 결측 대체 impute_median()
data_checks.py	결측률, 범위, 범주값, 중복, 분할 누수 검사
model.py	데이터 불러오기, 층화 분할, 모델 Pipeline
evaluate.py	지표 계산, 교차 검증
report.py	성능 리포트 자동 생성

github.com/leejaehee1/ai-quality-lab → day2/ 폴더

사람 대신 코드가 확인합니다

	수동 테스트	자동화 테스트
실행	사람이 매번 실행하고 눈으로 확인	코드가 실행하고 결과를 판정
속도	느리고 빠뜨리기 쉬움	몇 초 만에 수백 개를 똑같이
변경할 때	다시 하기 어려움	바꿀 때마다 자동 실행 (회귀 테스트)

회귀 테스트: 고친 뒤에도 기존 기능이 여전히 되는지 확인하는 것. 자동화의 가장 큰 가치입니다.

반복되고, 기준이 명확하고, 사람이 실수하기 쉬운 것

자동화한다

- 전처리 함수의 입력·출력
- 데이터 검증 (결측·범위·중복)
- 모델 성능 하한, 강건성
- 리포트 생성

사람이 한다

- 탐색적 테스트
- 사용성 평가
- 결과 해석과 출시 판단

ML은 데이터가 바뀌거나 재학습할 때마다 다시 확인해야 해서 자동화가 더 중요합니다.

1교시 · 저장소 준비

다섯 단계로 시작합니다

- 1 Fork**
ai-quality-lab을 내 계정으로 복사
- 3 Issues 켜기**
Settings → General → Features → Issues 체크
- 5 Colab**
노트북 01 열고 GITHUB_USER를 내 아이디로

- 2 Actions 허용**
Actions 탭 → 워크플로 실행 허용
- 4 웹 편집기**
저장소 화면에서 . 키 → github.dev

fork한 저장소는 Issues가 기본으로 꺼져 있습니다.

표준 라이브러리, 설치 필요 없음

```
import unittest
from mlqa.preprocess import is_eligible

class TestIsEligible(unittest.TestCase):           # ① TestCase 상속

    def test_typical_eligible(self):               # ② test_로 시작하는 메서드
        self.assertTrue(is_eligible(25, 12, False)) # ③ assert 메서드로 판정

    def test_invalid_age_raises(self):
        with self.assertRaises(ValueError):        # ④ 예외가 나야 통과
            is_eligible(200, 12, False)
```

판정과 준비

요소	역할
assertEqual, assertTrue, assertFalse	값 판정
assertRaises	예외 발생 확인
assertAlmostEqual	실수 비교
setUp()	테스트마다 먼저 실행되는 준비 코드
tearDown()	테스트마다 나중에 실행되는 정리 코드

```
PYTHONPATH=src python -m unittest discover -s tests -p "test_*unittest.py" -v
```

1교시 · 실습 8분

TestIsEligible에 테스트 두 개 추가

거주 기간 부족

`is_eligible(25, 3, False) → False`

취업 상태

`is_eligible(25, 12, True) → False`

이 두 테스트는 통과합니다. 결함은 아직 숨어 있습니다.

FIRST와 AAA

FIRST

- Fast: 빠르게
- Independent: 서로 독립적으로
- Repeatable: 언제 돌려도 같은 결과
- Self-validating: 스스로 판정
- Timely: 제때 작성

AAA 구조

- Arrange: 준비
- Act: 실행
- Assert: 확인

ML은 무작위성 때문에 Repeatable이 어렵습니다 → random_state 고정

PyTest는 unittest 테스트도 실행합니다

	unittest	PyTest
설치	표준 라이브러리	별도 설치
형태	클래스 + assert 메서드	함수 + 일반 assert 문
매개변수화	subTest (불편)	@pytest.mark.parametrize
준비 코드	setUp	fixture (재사용·범위 지정)

함수와 assert 문이면 충분합니다

```
def test_too_old():
    assert is_eligible(40, 12, False) is False

def test_rejects_string_age():
    with pytest.raises(ValueError):
        is_eligible("스물", 12, False)
```

명령	뜻
python -m pytest -q	간단히
-v	테스트 이름까지 자세히
-x	첫 실패에서 멈춤
-k 이름	이름으로 골라 실행

1일차 경계값 분석을 그대로 코드로

```
@pytest.mark.parametrize("age, expected", [
    (18, False), (19, True), (20, True),      # 경계 19의 3값
    (33, True), (34, True), (35, False),     # 경계 34의 3값
])
def test_age_boundaries(age, expected):
    assert is_eligible(age, 12, False) == expected
```

tests/test_my_checks.py에 작성하고 Colab 노트북 01에서 실행합니다.

2교시 · 실행 결과

18세에서 실패. 결함 1을 찾았습니다.

어제 슬라이드의 '커버리지 100%여도 남는 결함'과 같은 예입니다. 아직 고치지 말고 실패 메시지를 복사해 두세요.

어떤 입력에서, 무엇이 달랐나

```
FAILED tests/test_my_checks.py::test_age_boundaries[18-False] - assert True == False
```

[18-False]

매개변수 값. 어떤 입력에서 실패했는지 바로 보입니다.

assert True == False

실제 결과(True)가 기대(False)와 다릅니다.

고치기 전에 실패하는 테스트를 먼저 확보합니다. 고친 뒤 통과하면 확인 테스트가 됩니다.

준비 코드를 인자 이름으로 받아 씁니다

```
# tests/conftest.py
@pytest.fixture(scope="session")           # 전체에서 한 번만 (느린 준비)
def trained_model(data):                  # fixture가 fixture를 쓸 수도 있음
    X_train, _, y_train, _ = data
    return build_model().fit(X_train, y_train)

# tests/test_model.py
def test_prediction_shape_and_labels(trained_model, data):
    ...
```

기본 범위(function)는 테스트마다 새로 만듭니다. 데이터를 바꾸는 테스트에 씁니다.

pytest --cov=mlqa --cov-report=term-missing

Name	Stmts	Miss	Cover	Missing
src/mlqa/data_checks.py	15	0	100%	
src/mlqa/evaluate.py	15	0	100%	
src/mlqa/preprocess.py	16	0	100%	
src/mlqa/report.py	40	2	95%	88-89
TOTAL	99	2	98%	

preprocess.py는 커버리지 100%인데 결함 1이 있었습니다. 왜일까요?

모든 줄을 실행해도 경계값을 확인하지 않으면

커버리지가 알려 주는 것

어떤 줄이 한 번도 실행되지 않았는가 (Missing)

알려 주지 않는 것

실행한 줄이 올바르게 동작하는가. $18 \leq \text{age}$ 는 실행됐지만 틀렸습니다.

--cov-branch를 붙이면 분기 커버리지를 측정합니다.

지표 비교에는 approx

기능	예	용도
pytest.approx	assert 0.1 + 0.2 == pytest.approx(0.3)	실수 비교
tmp_path	generate_report(out_dir=tmp_path)	임시 폴더 파일 테스트
xfail	@pytest.mark.xfail(reason="이슈 #3")	알려진 결함 표시
마커	@pytest.mark.slow, -m "not slow"	느린 테스트 분리

0.1 + 0.2 == 0.3은 False입니다. 지표 값은 == 대신 approx로 비교합니다.

데이터 · 코드 · 모델

층	무엇을	이 저장소의 테스트
데이터 테스트	스키마, 결측, 범위, 범주, 중복, 분할 누수	test_data_checks.py
코드 테스트	전처리·지표 함수가 명세대로	test_preprocess*.py, test_evaluate.py
모델 테스트	성능 하한, 기준선 대비, 불변성, 강건성	test_model.py

1일차 7교시 검사를 assert로

```
def test_checks_find_injected_problems(dirty_df):  
    assert missing_rates(dirty_df)["radius"] == 0.2  
    assert out_of_range_count(dirty_df, RANGES)["radius"] == 1  
    assert invalid_category_count(dirty_df, "hospital", HOSPITALS) == 1  
    assert duplicate_count(dirty_df) == 1
```

깨끗한 데이터에서는 0건, 오류를 넣은 데이터에서는 정확한 건수. 검사 함수 자체도 검증합니다.

1일차 기준을 자동으로 확인

테스트	확인 내용	1일차
test_beats_baseline	기준선보다 정확도 0.1 이상 높음	3교시 기준선
test_cv_recall_meets_quality_gate	5겹 교차 검증 재현율 평균 0.90 이상	3교시 품질 기준
test_invariance_to_row_order	행 순서를 섞어도 예측이 같음	6교시 불변성
test_robust_to_small_noise	5% 잡음에서 재현율 하락 0.02 이하	6교시 섭동

기준은 상수로 맨 위에

```
MIN_CV_RECALL = 0.90

def test_cv_recall_meets_quality_gate(data):
    X_train, _, y_train, _ = data
    cv = cross_validate_model(build_model(), X_train, y_train)
    assert cv["recall"]["mean"] >= MIN_CV_RECALL, cv["recall"]
```

모델 테스트는 '정확히 이 값'이 아니라 '이 이상'을 확인합니다. 재학습하면 점수가 조금씩 바뀌기 때문입니다.

3교시 · 실습 15분

같은 파일에 테스트 두 개를 추가하세요

① 누수 테스트

`impute_median()`이 학습 데이터의 중앙값만 쓰는가?

힌트: 학습 `[1, 2, NaN, 4]`, 테스트 `[NaN, 100]`

② 지표 테스트

`compute_metrics()`의 정밀도가 손 계산과 같은가?

힌트: 두 클래스의 정밀도가 서로 다른 예를 만드세요

찾은 뒤에 원리를 확인합니다

누수

테스트 데이터의 값(100)이 중앙값 계산에 섞이면 테스트 정보가 전처리에 새어 들어갑니다. 1일차 5교시 전처리 누수.

양성 클래스

악성(0)을 찾는 모델인데 정밀도만 양성 종양(1) 기준으로 계산하면 보고서 숫자가 틀립니다. 1일차 3교시 pos_label.

테스트가 통과했다고 결함이 없는 것은 아닙니다. 예를 잘못 고르면 테스트도 결함을 놓칩니다.

반지름이 커지면 악성 확률이 줄면 안 된다

```
def test_bigger_radius_not_less_malignant(trained_model, data):
    _, X_test, _, _ = data
    X_big = X_test.copy()
    X_big[:, 0] *= 1.2                                # mean radius 1.2배
    p = trained_model.predict_proba(X_test)[:, 0]      # 악성 확률
    p_big = trained_model.predict_proba(X_big)[:, 0]
    assert (p_big >= p - 1e-9).mean() >= 0.95
```

실행해 보면 테스트 143건 모두 통과합니다.

4교시 · 한 번의 분할은 운이다

같은 모델, 분할만 바꿔 20번 평가 (노트북 02)

0.888

최저 테스트 정확도

0.958

최고 테스트 정확도

0.929

평균

점수 하나로 '모델 A가 B보다 낫다'고 말하기 어렵습니다.

모든 데이터가 한 번씩 검증에 쓰입니다

폴드 1: [검증] [학습] [학습] [학습] [학습]

폴드 2: [학습] [검증] [학습] [학습] [학습]

폴드 3: [학습] [학습] [검증] [학습] [학습]

폴드 4: [학습] [학습] [학습] [검증] [학습]

폴드 5: [학습] [학습] [학습] [학습] [검증] → 5개 점수의 평균 $\pm$ 표준편차

교차 검증은 학습 데이터 안에서 합니다. 테스트 데이터는 여전히 마지막에 한 번만.

라벨 순으로 정렬된 데이터, 폴드별 악성 비율 (원본 약 0.37)

폴드	KFold (섞지 않음)	StratifiedKFold
1	1.00	0.38
2	0.86	0.38
3	0.00	0.37
4	0.00	0.37
5	0.00	0.37

분류 문제에는 StratifiedKFold를 씁니다.

여러 지표를 한 번에

```
scoring = {
    "accuracy": "accuracy",
    "precision": make_scorer(precision_score, pos_label=0),    # 악성 = 양성 클래스
    "recall": make_scorer(recall_score, pos_label=0),
    "f1": make_scorer(f1_score, pos_label=0),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
result = cross_validate(model, X_train, y_train, cv=cv, scoring=scoring)
```

평균 ± 표준편차로 말합니다

지표	평균	표준편차
accuracy	0.979	0.009
precision	0.987	0.016
recall	0.956	0.025
f1	0.971	0.012

"재현율 0.956 ± 0.025 (5겹 층화 교차 검증, 학습 데이터 426건)"

4교시 · 모델은 Pipeline으로

교차 검증에서도 누수는 생깁니다

0.90

누수: 교차 검증 밖에서 특성 선택

1일차 무작위 데이터 실험을 교차 검증으로 다시 확인 (정답은 0.5 근처)

0.57

Pipeline: 폴드마다 학습 부분으로만

데이터의 구조에 맞게

방법	언제
GroupKFold	같은 환자의 데이터가 여러 개일 때 (중복 누수 방지)
TimeSeriesSplit	시간 순서가 있을 때 (과거로 학습, 미래로 검증)
K = 3 / 5·10 / 데이터 수	빠름·흔들림 큼 / 흔한 기본값 / 매우 느림

5교시 · 왜 자동으로 만드나

같은 형식으로 남겨야 비교할 수 있습니다

비교

재학습할 때마다 같은 형식의 결과

정확

손으로 복사하면 틀리고 빠뜨림

보관

CI가 매번 만들어 보관 (6교시)

PYTHONPATH=src python -m mlqa.report

단계	산출물
1. 학습 데이터로 5겹 교차 검증	지표별 평균·표준편차·폴드별 점수
2. 학습 후 테스트 데이터로 한 번 평가	테스트 지표, classification_report
3. 혼동 행렬 그림	reports/confusion_matrix.png
4. 기계가 읽는 결과	reports/metrics.json
5. 사람이 읽는 리포트	reports/model_report.md

matplotlib.use("Agg"): 화면 없는 서버(CI)에서도 그림을 파일로 저장

만드는 것보다 읽고 대조하는 것이 중요합니다

위치	악성 precision
표 2: 테스트 데이터 평가 (compute_metrics)	0.989
classification_report의 malignant 행	0.981

같은 모델, 같은 테스트 데이터인데 값이 다릅니다. 무엇을 의미할까요?

숫자가 서로 맞는지부터

- 같은 지표가 여러 곳에 있다면 값이 일치하는가
- 교차 검증 평균과 테스트 점수가 비슷한가 (크게 다르면 과적합·누수 의심)
- 튜는 폴드가 있는가
- 품질 기준(재현율 0.90 이상 등)을 만족하는가
- 데이터 건수, 양성 클래스 정의, 모델 설정이 적혀 있는가 (재현 가능성)

선택 5교시 · 리포트에 더 넣을 것

조별로 하나 골라 추가해 보기

항목	이유
기준선 모델 점수	얼마나 나은지
그룹별 지표	1일차 6교시 편향 점검
이전 버전과 비교	성능 저하(회귀) 감지
데이터 버전·커밋	재현 가능성
임계값별 정밀도·재현율	업무 판단 지원

올릴 때마다 자동으로 확인

용어	뜻
CI (지속적 통합)	코드를 올릴 때마다 자동으로 빌드·테스트
CD (지속적 배포·전달)	통과한 코드를 자동으로 배포(또는 배포 준비)
품질 게이트	기준을 통과하지 못하면 병합·배포를 막는 관문

ML에서는 성능 기준을 게이트에 넣어, 성능이 떨어진 모델이 배포되지 않게 막습니다.

.github/workflows/day2-ci.yml

```
on:                                # 언제: push, pull_request, 수동 실행
  push: { paths: ["day2/**"] }
  pull_request: { paths: ["day2/**"] }
jobs:
  test:
    runs-on: ubuntu-latest          # 어디서: GitHub 가상 머신
    steps:                           # 무엇을: 차례대로
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: pytest --cov=mlqa --cov-report=term-missing
      - run: python -m mlqa.report    # if: always()
      - uses: actions/upload-artifact@v4
```

다섯 개만 알면 읽을 수 있습니다

용어	뜻
workflow	자동화 전체 (YAML 파일 하나)
event (on)	실행 계기: push, pull_request, 수동 실행
job	가상 머신 하나에서 실행되는 작업 묶음
step	job 안의 한 단계: uses(만들어진 액션) 또는 run(명령)
artifact	실행 결과 파일 보관

if: always() → 테스트가 실패해도 리포트는 생성합니다. 실패했을 때 리포트가 더 필요하니까요.

6교시 · 실습 15분

빨간 불을 켜 봅시다

- 1 테스트 커밋**
github.dev에서 tests/test_my_checks.py 추가 후 커밋
- 3 로그 읽기**
실패한 step에서 어떤 테스트가 실패했나

- 2 Actions 확인**
방금 실행된 워크플로의 빨간 X
- 4 리포트 보기**
실행 요약(Summary), Artifacts의 model-report

병합하면 안 되는 상태

PR 화면

체크 결과가 표시되어 리뷰어가 바로 봅니다.

브랜치 보호 규칙

'상태 확인 통과 시에만 병합'을 켜면 빨간 불인 PR은 병합 버튼이 막힙니다. 이것이 품질 게이트입니다.

외부 텍스트를 명령에 넣지 않습니다

이슈 제목, PR 본문처럼 외부 사람이 쓴 텍스트를 `run:` 명령에 그대로 넣으면 명령어 주입 공격을 당할 수 있습니다. 필요하다면 `env`로 받아 따옴표로 감싸 씁니다. 오늘 워크플로는 그런 값을 쓰지 않습니다.

CI를 더 쓸모 있게

주제	내용
매트릭스 빌드	여러 Python 버전에서 동시에 테스트
캐시	pip 설치 시간 단축 (이미 적용)
배지	README에 CI 상태 표시
예약 실행	매일 밤 재학습·평가 (드리프트 감지)
CD	통과 시 모델 파일 자동 배포

누가 읽어도 똑같이 재현할 수 있게

구성	좋은 예	나쁜 예
요약	is_eligible()이 만 18세를 지원 대상으로 판정함	판정 함수 이상함
재현 절차	입력값과 명령을 번호 순서로	해 보면 틀려요
기대 결과	False (요구사항: 만 19세 이상)	없음
실제 결과	True, 실패한 테스트 로그	이상하게 나옴
근거·환경	테스트 이름, CI 실행 링크, 커밋	없음

사실과 추측을 구분합니다. 원인 추정은 따로 적습니다.

영향과 긴급함은 다를 수 있습니다

	심각도 (Severity)	우선순위 (Priority)
질문	영향이 얼마나 큰가	얼마나 빨리 고쳐야 하나
주로 정하는 사람	테스터	관리자, 제품 책임자
예: 첫 화면 회사명 오타	낮음	높음
예: 거의 안 쓰는 기능 멈춤	높음	낮음

토론: 오늘 찾은 결함 3개의 심각도와 우선순위는?

Issues → New issue → 버그 리포트 양식

양식이 요구하는 것

- 요약
- 심각도 · 우선순위 (선택 목록)
- 재현 절차
- 기대 결과 · 실제 결과
- 근거 (실패한 테스트, CI 링크)

규칙

- 결함마다 이슈 하나 (총 3개)
- 필수 칸을 비우면 등록 안 됨
- bug 라벨 자동

결함 하나에 PR 하나

테스트 실패 → 이슈 등록 → 브랜치에서 수정 → PR 본문에 Fixes #1 → CI 초록 불 → 병합 → 이슈 자동 종료

1

브랜치

fix/age-boundary

3

CI 확인

추가한 테스트 통과(확인), 기존 테스트 통과(회귀)

2

PR

본문에 Fixes #이슈번호

4

병합

연결된 이슈가 자동으로 닫힘

7교시 · 결함 1만 고쳤는데 빨간 불?

다른 결함의 테스트가 아직 실패하니깐요

방법 1

세 결함을 차례로 고치고 PR을 하나씩 병합

방법 2

아직 안 고친 결함의 테스트에

@pytest.mark.xfail(reason="이슈 #2")를 붙여 두기

어떤 방법을 택했는지 발표에서 설명해 주세요.

GitHub 외의 도구도 구성은 비슷합니다

기능	용도
라벨	bug, enhancement, 심각도별 분류·검색
담당자	누가 처리하는지
마일스톤	배포 버전 단위로 묶기
프로젝트 보드	할 일 → 진행 중 → 완료
이슈 템플릿	필수 정보가 빠지지 않게 (오늘 사용)

Jira, Redmine 등도 같은 개념을 씁니다.

8교시 · 조별 발표

조당 3~4분

- 1** **찾은 결함**
어떤 테스트로 찾았나
- 3** **이슈 · PR**
링크와 심각도·우선순위 판단

- 2** **CI 결과**
빨간 불 → 초록 불
- 4** **배운 점**
다음 프로젝트에 적용할 한 가지

기준을 이해하고, 자동화했습니다

1일차 · 기준

- 테스트 원칙, 블랙박스·화이트박스
- 분류·회귀 지표와 해석
- 과적합·누수·편향·강건성
- 데이터 품질 검수 기준

2일차 · 자동화

- unittest · PyTest
- 경계값·누수·지표 테스트로 결함 3개
- 교차 검증과 성능 리포트
- CI 품질 게이트, 이슈 · PR

생성형 AI 서비스는 무엇이 더 필요할까

- 정답이 하나가 아님 → LLM 채점(LLM-as-a-Judge), 골든셋
- 악용 가능성 → 레드팀 테스트 (프롬프트 인젝션 등)
- 운영 비용 → 응답시간·토큰 비용 측정
- 실습 자료: 저장소 [appendix/llm-service](#) (promptfoo, DeepEval)

마지막 메시지

테스트가 통과했다는 것은 결함이 없다는 뜻이 아닙니다

무엇을 기준으로 테스트했는지가 품질을 결정합니다.